Java: ANNOTATIONS

"Concept && Coding" YT Video Notes

What is Annotation?

- It is kind of adding **META-DATA** to the Java code.
- Always start having **@** symbol.
- We can use this meta data information at **runtime** and can add certain logic in our code if warrnot.
- How to read Meta data information: Using **Reflection** or discussed in previous video.
- Annotations can be applied at anywhere like Classes, Methods, Interface, fields, parameters etc.

Example:

```
public interface Bird {
    public boolean fly();
}

//Annotation phrased simply:
public class Jaiju implements Bird
    @Override
    public boolean fly() {
        return true;
    }
}
```

TYPES OF ANNOTATIONS:

- Pre-Defined Annotations
  - Used on Annotations (Called Meta Annotations)
    - @Target
    - @Retention
    - @Documented
    - @Inherited
    - @Repeatable (Not Shown)
  - Used on Java code (In Classes, Methods etc.)
    - @Deprecated
    - @Override
    - @SuppressWarnings
    - @FunctionalInterface
    - @SafeVarargs
- Custom Annotations / User Defined (Our custom name)

Annotations used on Java Code:

@Override:

- During Compile time, it will check that the method should be Override.
- Can be used over **Method**, **Interface**, if it did not match with the parent method.
- Can be used over **Retention**.

```
public interface Bird {
    public boolean fly();
}

public class Jaiju implements Bird
    @Override
    public boolean fly() {
        return true;
    }
}
```

@SuppressWarnings:

- Can be used over **Method** or **CONSTRUCTOR** (Any single time) **BEGINNING**
- Stop it getting used during runtime.
- Can be used over **Field**, **Method**, **Parameter**, **Constructor**, **Local Variable**, **Type** (Class or Interface or enum)

```
class Test {
    @SuppressWarnings("deprecation")
    public static void main(String[] ...) {
        Bird b = new Jaiju();
        boolean result = b.fly();
        System.out.println(result);
    }
}
```

```
class Test {
    @SuppressWarnings("unchecked")
    public static void main(String[] ...) {
        Bird b = new Jaiju();
        boolean result = b.fly();
        System.out.println(result);
    }
}
```

```
class Test {
    @SuppressWarnings("deprecation")
    public static void main(String[] ...) {
        Bird b = new Jaiju();
        boolean result = b.fly();
        System.out.println(result);
    }
}
```

OR

```
class Test {
    @SuppressWarnings("deprecation", "unchecked")
    public static void main(String[] ...) {
        Bird b = new Jaiju();
        boolean result = b.fly();
        System.out.println(result);
    }
}
```

OR

```
class Test {
    @SuppressWarnings("deprecation", "unchecked", "rawtypes")
    public static void main(String[] ...) {
        Bird b = new Jaiju();
        boolean result = b.fly();
        System.out.println(result);
    }
}
```

@FunctionalInterface:

- *Restrict Interface to have only 1 abstract method*
- Throws Compilation error, if more than 1 abstract method found.
- Can be used over **Type** (Class or Interface or enum)

```
@FunctionalInterface
public interface Bird {
    public boolean fly();
    public void eat();
}
```

@Override:

- **Retention**: **SOURCE** ❌ **CLASS** ❌ **RUNTIME** ✅
- **Target**: **METHOD**, **TYPE**, **CONSTRUCTOR**, **FIELD**, **PARAMETER**, **TYPE_USE**, **TYPE_PARAMETER**, **MODULE**, **RECORD_COMPONENT**, **PROVIDES**, **USES**, **EXPORTS**, **OPENS**, **MODULE**
- **Inherited**: ❌
- **Documented**: ❌
- **Repeatable**: ❌

What is Bug Fixation?

Return of the Type Cast Error Bridge, caused by the reference of Number type Object Creation Image!!

```
class Test {
    public static void main(String[] ...) {
        Number[] ns = new Integer[10];
        ns[0] = new Integer(0);
        ns[1] = new Double(1.0);
        System.out.println(ns[0]);
        System.out.println(ns[1]);
    }
}
```

```
class Test {
    public static void main(String[] ...) {
        Number[] ns = new Integer[10];
        ns[0] = new Integer(0);
        ns[1] = new Double(1.0);
        System.out.println(ns[0]);
        System.out.println(ns[1]);
        System.out.println(ns[2]);
    }
}
```

Annotations used over Another Annotations (META-ANNOTATIONS):

@Target:

- This meta-annotation will restrict, where to use the annotation.
  Either at method or Constructor or fields etc.

```
@Target(ElementType.METHOD)
public @interface Deprecated {
public @interface Override {

@Target({ElementType.CONSTRUCTOR, ElementType.METHOD})
public @interface SafeVarargs {

Element Type:
TYPE
FIELD
METHOD
PARAMETER
CONSTRUCTOR
LOCAL_VARIABLE
ANNOTATION_TYPE
PACKAGE
TYPE_PARAMETER (Allow you to apply on generic types <T>)
TYPE_USE (Java 8 Feature, Allow you to use annotation of all place where Type you can declare.Like: List @Annotation String>
```

@Retention:

- This meta-annotation tells, how Annotation will be stored in java.
  - **RetentionPolicy.SOURCE**: Annotations will be discarded by the compiler itself and it will not be recorded in .class file
  - **RetentionPolicy.CLASS**: Annotations will be recorded in .class file but will be ignore by JVM at run time.
  - **RetentionPolicy.RUNTIME**: Annotations will be recorded in .class file + available during run time. Usage of reflection can be done.

```
class Test {
    public static void main(String[] ...) {
        Bird b = new Jaiju();
        boolean result = b.fly();
        System.out.println(result);
    }
}
```

```
class Test {
    public static void main(String[] ...) {
        Bird b = new Jaiju();
        boolean result = b.fly();
        System.out.println(result);
    }
}
```

```
class Test {
    public static void main(String[] ...) {
        Bird b = new Jaiju();
        boolean result = b.fly();
        System.out.println(result);
    }
}
```

@Documented:

- Marks the annotation as a special one, such that Documentation is generated for this type when annotation is used in Source code.

NOT DOCUMENTED:

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface MyAnn {

public class Jaiju implements Bird {
    @Override
    public boolean fly() {
        return true;
    }
}
```

```
class Test {
    public static void main(String[] ...) {
        Bird b = new Jaiju();
        boolean result = b.fly();
        System.out.println(result);
    }
}
```

```
class Test {
    public static void main(String[] ...) {
        Bird b = new Jaiju();
        boolean result = b.fly();
        System.out.println(result);
    }
}
```

```
class Test {
    public static void main(String[] ...) {
        Bird b = new Jaiju();
        boolean result = b.fly();
        System.out.println(result);
    }
}
```

@Inherited:

- Marks the annotation as a special one, such that Documentation is generated for this type when annotation is used in Source code.
- Both this meta-annotation and a Documented will cause it to be recorded.

NOT DOCUMENTED:

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface MyAnn {

@MyAnn
public class Jaiju implements Bird {
    @Override
    public boolean fly() {
        return true;
    }
}
```

```
class Test {
    public static void main(String[] ...) {
        Bird b = new Jaiju();
        boolean result = b.fly();
        System.out.println(result);
    }
}
```

```
class Test {
    public static void main(String[] ...) {
        Bird b = new Jaiju();
        boolean result = b.fly();
        System.out.println(result);
    }
}
```

```
class Test {
    public static void main(String[] ...) {
        Bird b = new Jaiju();
        boolean result = b.fly();
        System.out.println(result);
    }
}
```

@Repeatable:

- Marks an annotation as a special one, such that it can be used multiple times on the same Source code.
- Only Allowed in Java 8 Version, if Annotation is used more than a single time.

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface MyAnn {

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface MyAnnContainer {
    MyAnn[] value();
}
```

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface MyAnn {

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface MyAnnContainer {
    MyAnn[] value();
}
```

Output:

```
Process finished with exit code 0
```

Output:

```
Process finished with exit code 0
```

Repeatable:

- Marks an annotation as a special one, such that it can be used multiple times on the same source code.

We can use it on the below code:

```
@Repeatable
@Deprecated
public class Jaiju {
    public void fly() {

    }
}
```

```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
public @interface Author {
    String name();
}

@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
public class Jaiju {
    public void fly() {

    }
}
```

We need to use @Repeatable Meta annotation

```
@RepeatableAnnotationTypes.class(
@interface AnnotationTypes {
    String[] value();
}

@RepeatableAnnotationTypes.class(
@interface RepeatableAnnotationTypes {
    AnnotationTypes[] value();
}
```

```
@RepeatableAnnotationTypes.class(
@interface AnnotationTypes {
    String[] value();
}

@RepeatableAnnotationTypes.class(
@interface RepeatableAnnotationTypes {
    AnnotationTypes[] value();
}

@AnnotationTypes(value = "Hello")
@AnnotationTypes(value = "HelloAgainOnce")
@AnnotationTypes(value = "HelloAgainOnce")
public class Jaiju {
    public void fly() {

    }
}
```

```
class Test {
    public static void main(String[] ...) {
        AnnotationTypes[] annotationTypes = (AnnotationTypes[]) getAnnotation(Jaiju.class, RepeatableAnnotationTypes.class);
        System.out.println("Length: " + annotationTypes.length);
        for (AnnotationTypes annotationType : annotationTypes) {
            System.out.println(annotationType.value());
        }
    }
}
```

Output:

```
D:\IntelliJ Proj
Length: 3
HelloAgainOnce
HelloAgainOnce
Hello

Process finished with exit code 0
```

User Defined or Custom Annotations:

- **We can create our own ANNOTATION using Keyword "@interface"**

Creating an Annotation with empty Body:

```
public @interface MyCustomAnnotation {

    @MyCustomAnnotation
    public class Jaiju {
        public void fly() {

        }
    }
}
```

Creating an Annotation with method (No more like a field):

- No parameter, no body.
- Return type is restricted to Primitive, Class, String, enums, annotations and array of these types.

```
public @interface MyCustomAnnotation {

    String name();
}

@MyCustomAnnotation(name = "Testing")
public class Jaiju {
    public void fly() {

    }
}
```

Creating an Annotation with an element with Default values:

- **Default value can not be null**

```
public @interface MyCustomAnnotation {

    String name() default "hello";
}

@MyCustomAnnotation
public class Jaiju {
    public void fly() {

    }
}
```